

HW06 — API Testing Report

Student ID: 23127531

Repository: <https://github.com/aohymevoL666/software-testing-homework6>

SUT Base URL: <http://localhost:3000>

Testing tools: Postman, Newman, GitHub Actions, ChatGPT

AI model used: GPT-5.6 Sol

1. Objective and Scope

This homework applies an AI-first API-testing workflow to three backend APIs. For each selected API, the workflow is:

1. Generate at least 35 candidate test cases with AI.
2. Human-audit every AI case as VALID / INVALID / INCOMPLETE.
3. Add at least five extension cases.
4. Execute the reviewed suite with Postman/Newman.
5. Report genuine defects with evidence and GitHub Issues.

Every automated request uses the required **X-Student-Id: 23127531** header.

2. Selected APIs

Pool	Feature	Endpoint
A	FR-01 Account Registration	POST /api/register
B	FR-07 Shopping Cart	POST /api/cart
C	FR-12 Access Control	GET /api/admin/users

3. Overall Test Summary

Metric	API 1	API 2	API 3	Total
AI-generated cases	42	42	42	126
VALID after audit	11	13	26	50
INVALID after audit	5	2	0	7
INCOMPLETE after audit	26	27	16	69
Human-added cases	6	6	6	18
Total logical cases	48	48	48	144
Logical cases executed	48	48	35	131
Logical PASS	46	48	27	121
Logical FAIL	2	0	8	10
Confirmed bugs	1	0	3	4

The consolidated workbook is `test-cases/HW06_Test_Cases_and_Summary_23127531.xlsx`.

4. API 1 — FR-01 Account Registration

4.1 Generate

AI generated 42 registration cases covering:

- name/email/password partitions;
- missing/null/body variants;
- duplicate-registration state;
- security-oriented strings and unexpected fields;
- response schema.

4.2 Human Audit

Audit result:

- VALID: 11
- INVALID: 5
- INCOMPLETE: 26

The main correction was to reject unsupported assumptions. The implementation did not define strict email-format validation, password-complexity rules, or email uniqueness. Those behaviors therefore could not be treated as specification-backed failure oracles.

4.3 Extend

Six extension cases were retained, including:

- `text/plain` registration;
- missing `Content-Type`;
- null body;
- duplicate email with different credentials;
- email case variation;
- mass assignment of sensitive fields.

4.4 Execution

Newman execution:

- Requests: 53
- Assertions: 134
- Failed assertions: 2

The two failures were `REG-HUM-001` and `REG-HUM-002`, both returning HTTP 500 for unsupported/missing `Content-Type` handling.

4.5 Confirmed Bug

Issue #1 — Unsupported or missing `Content-Type` causes HTTP 500

GitHub Issue:

<https://github.com/aohymevoL666/software-testing-homework6/issues/1>

The two variants are treated as one root-cause defect.

5. API 2 — FR-07 Shopping Cart

5.1 Generate

AI generated 42 cart cases covering:

- id/name/price/quantity partitions;
- authentication;
- malformed JSON;
- repeated-add state;
- user isolation;
- response and payload robustness.

5.2 Human Audit

Audit result:

- VALID: 13
- INVALID: 2
- INCOMPLETE: 27

Two important AI assumptions were rejected:

- nonexistent product ID must return 404;
- repeated adds must merge quantity.

Neither behavior was specified by the supplied API contract.

5.3 Extend

Six extension cases examined:

- contradictory metadata across repeated requests;
- two-user isolation;
- stale but valid JWT characterization;
- nested-object payloads;
- downstream persistence of unexpected fields;
- cart state after backend restart.

5.4 Automated Execution

Newman execution:

- Requests: 62
- Assertions: 109
- Failed assertions: 0

- Duration: approximately 5.5 seconds

All automated corrected/characterization oracles passed.

5.5 Manual State Test

CART-HUM-006 verified process-memory persistence.

Before backend restart:

```
[{"id":990006,"name":"RESTART_MARKER","price":606,"quantity":1}]
```

After restart and re-login using the seeded account:

```
[]
```

This was classified as a characterization result, not a confirmed bug, because the API specification does not explicitly require persistence across server restarts.

5.6 Confirmed Bugs

0 confirmed bugs.

6. API 3 — FR-12 Access Control

6.1 Generate

AI generated 42 cases covering:

- valid/invalid JWT;
- Admin vs normal-user role;
- wrong authentication schemes;
- role transitions;
- privilege escalation;
- response disclosure;
- unsupported HTTP methods.

6.2 Human Audit

Audit result:

- VALID: 26
- INVALID: 0
- INCOMPLETE: 16

The explicit Admin-only requirement provided a strong oracle for normal-user denial. In contrast, token revocation after account deletion, demotion, or logout was not explicitly defined and was therefore retained as characterization rather than mandatory behavior.

6.3 Extend

Six extension cases focused on:

- direct normal-user access to the global user list;
- full self-promotion flow through PUT `/api/users/me`;
- PII disclosure in unauthorized responses;
- valid JWT under the wrong authorization scheme;
- stale privilege characterization;
- response privacy.

6.4 Execution

Newman execution:

- Requests: 47
- Assertions: 57
- Failed assertions: 18
- Duration: approximately 4.2 seconds

The 18 assertion failures reduced to three genuine root-cause defects plus one non-bug oracle correction. At the logical-case level, eight executed cases exposed the three defects.

6.5 Confirmed Bugs

Issue #2 — Broken Admin Access Control

A normal authenticated user receives 200 OK and the global user list from GET `/api/admin/users`.

<https://github.com/aohymevoL666/software-testing-homework6/issues/2>

Issue #3 — Bearer Scheme Not Enforced

A valid JWT supplied as:

Authorization: Basic <valid-jwt>

is accepted even though the documented scheme is Bearer.

<https://github.com/aohymevoL666/software-testing-homework6/issues/3>

Issue #4 — Vertical Privilege Escalation

A normal user can submit `role:"admin"` to PUT `/api/users/me`, re-login, and obtain Admin privileges.

<https://github.com/aohymevoL666/software-testing-homework6/issues/4>

7. Postman Features Used

The work exercised the following Postman/Newman features:

- Collections;
- environments;
- environment variables;
- dynamic runtime variables;
- collection-level pre-request scripts;
- automatic **X-Student-Id** header injection;
- test scripts and assertions;
- multi-request setup/state-validation flows;
- Postman Console;
- Newman CLI;
- Newman HTML reporting;
- execution from GitHub Actions.

Mocks, monitors, and data-file-driven Collection Runner runs were not used because they were not necessary for the selected APIs.

8. X-Student-Id Evidence

A Postman Console screenshot demonstrates an actual request containing:

X-Student-Id: 23127531

Evidence:

evidence/postman_console_student_id.png

The automated collection-level pre-request scripts also log the header value for each executed request.

9. CI/CD Integration

GitHub Actions workflow:

`.github/workflows/api-tests.yml`

The pipeline:

1. checks out the HW06 repository;
2. clones the eShop SUT;
3. installs backend dependencies;
4. starts the backend;
5. waits for `/api/products`;
6. installs Newman;
7. runs the API 2 regression collection;
8. runs a deterministic CI smoke assertion.

Sample PASS run

Commit:

170ab50 - ci: add passing API test workflow

Result: PASS.

Intentional FAIL run

Commit:

31fa261 - ci: demonstrate failing API assertion

The smoke oracle was intentionally changed from expected HTTP 200 to 201.
The API still returned 200, so the workflow correctly failed.

Restored final state

Commit:

916e0ff - ci: restore passing state and record CI evidence

Evidence:

- evidence/ci_pass_run.png
- evidence/ci_fail_run.png
- docs/ci_cd_report.md

10. Agent Skill — AI-Driven API Test Generator

The designed generator accepts an API specification and produces candidate tests across:

- happy paths;
- partitions and boundaries;
- authentication;
- authorization;
- state;
- security;
- protocol variations;
- response schema and sensitive-data checks.

A mandatory Human Audit stage labels each case VALID / INVALID / IN-COMplete before execution.

Artifacts:

- agent-skill/HW06_API_Test_Generator_Design.md
- agent-skill/api_test_generator_pseudocode.txt
- agent-skill/api_test_generator_diagram.png

The diagram was drawn manually by the student.

11. Key Lessons

The largest AI failure mode was treating common industry best practices as if they were explicit SUT requirements. This appeared in registration validation, cart product validation, duplicate behavior, and token-revocation assumptions.

Human review was therefore used to separate:

- explicit requirement-backed oracles;
- useful but under-specified characterization tests;
- invalid AI assumptions.

The strongest defects were found when the specification provided a clear oracle, especially the Admin-only access-control requirement.

12. Submission Artifact Map

Requirement	Artifact
Main report	reports/HW06_Report.md + PDF
Public repository	https://github.com/aohymevoL666/software-testing-homework
Postman collections	postman/*.postman_collection.json
Newman reports	reports/newman/*.html
Postman features list	Section 7 of this report
CI/CD report	docs/ci_cd_report.md
Excel test cases/summary	test-cases/HW06_Test_Cases_and_Summary_23127531.xlsx
Generator diagram	agent-skill/api_test_generator_diagram.png
Generator pseudocode	agent-skill/api_test_generator_pseudocode.txt
Bug reports	reports/bugs/
GitHub Issue links	docs/github_issues.md
AI Critique	audit/HW06_AI_Critique.md
AI Audit	audit/HW06_AI_Audit.md
Git commit log	git-commit-log.txt
README / self-assessment	README.md

13. Self-Assessment

Criterion	Maximum	Self-Assessed
API 1	30	29
API 2	30	29
API 3	30	27
Agent Skill	10	10
Total	100	95

The self-assessment is intentionally conservative because 13 retained API 3 cases

were not selected for execution and exact clock times for every AI interaction were not preserved.

Appendix A — AI Critique

HW06 — AI Critique

Student ID: 23127531

Length: approximately 250 words

AI was useful for accelerating API test design, especially when generating broad input partitions, authentication variants, response checks, and state-transition ideas. It also reduced repetitive work by producing Postman collections, Newman-oriented assertions, CI configuration, bug-report templates, and spreadsheet summaries. However, the strongest lesson from this homework is that AI-generated tests are only starting points, not trustworthy test oracles.

For the registration API, AI initially assumed common production rules such as valid email syntax, password constraints, and duplicate-email rejection. Those expectations were not supported by the supplied specification or implementation, so human review had to mark many cases INVALID or INCOMPLETE. A similar issue appeared in the cart API, where AI assumed nonexistent product IDs should return 404 and repeated additions should merge quantities. Source inspection showed that the implementation simply stores the submitted body in memory, making those expectations unjustified.

AI performed better when the requirement was explicit. For the Admin users API, the specification clearly required an Admin account, so the generated non-admin denial tests had a reliable oracle. Keeping those assertions strict exposed a genuine broken-access-control defect. Human review was still necessary to separate that defect from weaker assumptions about JWT revocation, logout, stale roles, or exact response schemas.

Overall, AI improved speed, breadth, documentation consistency, and automation, but it also tended to import “best practice” expectations that were not actual requirements. The most valuable human contribution was validating every oracle against the specification and source, then distinguishing confirmed defects from robustness observations and characterization results.

Appendix B — AI Audit Report

HW06 — AI Audit Report

Student ID: 23127531

Assignment: HW06 — API Testing

AI tool: ChatGPT

Model: GPT-5.6 Sol

Primary usage dates: 2026-09-02 to 2026-09-03

Local timezone: UTC+07:00

Repository: <https://github.com/aohymevoL666/software-testing-homework6>

Timestamp note: The retained project transcript reliably preserves the interaction dates and order, but not an exact clock time for every individual AI exchange. Before final submission, exact times can be copied from the ChatGPT conversation UI if the instructor requires per-message clock timestamps. The audit below does not invent missing times.

AI usage policy

AI was used for test ideation, audit support, Postman/Newman artifact generation, execution-result interpretation, CI/CD scaffolding, spreadsheet consolidation, and documentation drafting. AI output was not accepted blindly: generated cases were compared with the supplied API specification and inspected SUT implementation, then classified as VALID / INVALID / INCOMPLETE before execution.

Interaction log

#	Date	Time	User prompt / interaction summary	AI output / assistance	Human review / action
1	2026-09-02	Not preserved	Establish HW06 scope, environment, and select one API from each pool.	Proposed and retained the baseline: POST <code>/api/register</code> , POST <code>/api/cart</code> , GET <code>/api/admin/users</code> ; organized repository structure and execution plan.	Student accepted the three-API baseline and generated the workspace.
2	2026-09-02	Not preserved	Inspect API 1 registration specification and implementation.	Identified request fields, response behavior, database schema, and missing validation constraints.	Student verified source and backend availability.
3	2026-09-02	Not preserved	Generate registration API tests with broad partition/security/schema coverage.	Generated 42 AI test cases in <code>api1_registration</code>	Student saved and committed <code>api1_registration_ai_generated.md</code> . AI-generated artifact.

#	Date	Time	User prompt / interaction summary	AI output / assistance	Human review / action
4	2026-09-02	Not preserved	Audit all AI-generated registration cases against the actual specification/implementation.	Classified 42 cases as 11 VALID, 5 INVALID, 26 INCOMPLETE; explained fabricated validation/uniqueness assumptions.	Student saved and committed the audit.
5	2026-09-02	Not preserved	Extend API 1 coverage after the audit.	Proposed six additional candidate cases covering Content-Type handling, null body, duplicate/case-variant email, and mass assignment.	Student reviewed, accepted, saved, and committed the extension.

#	Date	Time	User prompt / interaction summary	AI output / assistance	Human review / action
6	2026-09-02	Not preserved	Implement and execute API 1 in Postman/Newman	Generated Postman collection/environment added level X-Student-Id and interpreted Newman results, and drafted execution/bug artifacts.	Student executed Newman; about 500 responses were confirmed and evidence was captured.
7	2026-09-02	Not preserved	Inspect API 2 shopping-cart implementation.	Identified bearer authentication, in-memory userCarts , and absence of payload/product validation.	Student confirmed source and retained FR-07 as API 2.

#	Date	Time	User prompt / interaction summary	AI output / assistance	Human review / action
8	2026-09-02	Not preserved	Generate and audit API 2 cases.	Generated 42 cases, then audited them as 13 VALID, 2 INVALID, 27 INCOMPLETE.	Student saved and committed both artifacts.
9	2026-09-02	Not preserved	Extend API 2 after AI audit.	Proposed six candidate state/security cases: metadata inconsistency, user isolation, stale JWT characterization, nested payloads, extra-field persistence, and restart persistence.	Student reviewed, accepted, saved, and committed the extension.

			User prompt / interac- tion summary	AI output / assistance	Human review / action
#	Date	Time			
10	2026-09-02	Not preserved	Implement API 2 Postman collection and execute it.	Generated 62-request collection plus environment; automated two-user setup and 47 logical cases, leaving restart persistence for manual execution.	Student ran Newman: 109/109 assertions passed.
11	2026-09-03	Not preserved	Complete manual cart restart-state test.	Diagnosed an invalid first attempt caused by an empty email and backend reseeding; then proposed a seeded-user rerun.	Student reran using test@eshop.com ; cart contained marker before restart and [] after restart. Classified as characterization, not a spec defect.

#	Date	Time	User prompt / interaction summary	AI output / assistance	Human review / action
12	2026-09-03	Not preserved	Inspect API 3 Admin users route and authorization middle-ware.	Found that GET <code>/api/admin/users</code> uses <code>authenticate</code> only and performs no Admin role check; identified response fields.	Student accepted the execution of the code.
13	2026-09-03	Not preserved	Generate API 3 access-control tests.	Generated 42 AI cases covering auth, authorization, role claims, stale state, privilege escalation, response schema, and protocol handling.	Student saved and committed the generation artifact.

			User prompt / interac- tion summary	AI output / assistance	Human review / action
#	Date	Time			
14	2026-09-03	Not preserved	Audit all API 3 AI cases.	Classified 26 VALID and 16 INCOMPLETE; separated explicit Admin-only requirements from unsupported token-revocation assumptions.	Student saved and committed the audit.
15	2026-09-03	Not preserved	Extend API 3 coverage.	Proposed six candidate cases emphasizing direct non-admin access, end-to-end self-promotion, PII disclosure, wrong auth scheme, stale privilege, and response privacy.	Student reviewed, accepted, saved, and committed the extension.

#	Date	Time	User prompt / interaction summary	AI output / assistance	Human review / action
16	2026-09-03	Not preserved	Implement and execute API 3 in Postman/Newman.	Generated 47-request collection; retained strict FR-12 assertions rather than forcing a green run. Interpreted 18 failures into three genuine bugs plus one oracle correction.	Student executed Newman and captured evidence.
17	2026-09-03	Not preserved	Draft API 3 bug reports.	Produced Mark-down reports for broken Admin access control, Bearer-scheme enforcement, and vertical privilege escalation.	Student committed reports and created GitHub Issues #2-#4.

#	Date	Time	User prompt / interaction summary	AI output / assistance	Human review / action
18	2026-09-03	Not preserved	Publish repository and document confirmed bugs as GitHub Issues.	Provided issue titles/bodies and a linking artifact.	Student created four public GitHub Issues and committed <code>docs/github_issues.md</code> .
19	2026-09-03	Not preserved	Add CI/CD evidence with one passing and one intentional failing test.	Generated GitHub Actions workflow, deterministic CI smoke collection/environment and instructions for PASS → intentional FAIL → restore PASS.	Student produced green commit <code>170ab50</code> , failing commit <code>31fa261</code> , and restored passing state in <code>916e0ff</code> .
20	2026-09-03	Not preserved	Capture anti-cheat evidence in Postman Console.	Gave step-by-step UI instructions, diagnosed <code>{{baseurl}}</code> vs <code>{{baseUrl}}</code> , and verified the final screenshot.	Student captured and committed <code>evidence/postman_console_student_i</code>

#	Date	Time	User prompt / interaction summary	AI output / assistance	Human review / action
21	2026-09-03	Not preserved	Consolidate test cases and summary into the required spreadsheet artifact.	Generated HW06_Test_Cases and Summary_23127531.xlsx with three API sheets, and Bug Register.	Student committed workbook in 402ac63.
22	2026-09-03	Not preserved	Prepare mandatory AI audit and critique documentation.	Generated this audit and the separate 200–300 word critique.	Student must review wording and fill exact clock times only if required by the instructor.

Important human-audit examples

Example 1 — API 1 validation assumptions

AI initially proposed expectations such as strict email-format validation, password rules, and duplicate-email rejection. Human/source review found that the supplied implementation and database schema did not define those constraints. Those cases were therefore marked INVALID or INCOMPLETE rather than being executed with fabricated oracles.

Example 2 — API 2 product/cart semantics

AI initially assumed that a nonexistent product id should return 404 and that repeated adds should merge quantities. Source review showed that `POST /api/cart` simply pushes the request body into an in-memory array. These assumptions were rejected during human audit.

Example 3 — API 3 authorization

The Admin-only rule was explicitly supported by the supplied API specification. Therefore normal-user denial cases remained VALID even though source inspection predicted they would fail. Execution later confirmed the defect: normal users received 200 OK and the global user list.

Disclosure about the “human extension” artifacts

AI assisted with proposing and formatting candidate extension cases after the audit. The student reviewed, selected, executed, and accepted the final cases. Because the assignment wording asks for student-added cases that AI missed, the safest academic-integrity approach is for the student to independently revise the wording or substitute at least five personally authored extension cases per API before final submission if the instructor interprets “of your own” as prohibiting AI-assisted ideation.

Files produced with AI assistance

- test-cases/api1_registration_ai_generated.md
- test-cases/api1_registration_audit.md
- test-cases/api1_registration_human_extension.md
- postman/HW06_API1_Registration.postman_collection.json
- docs/api1_registration_execution_summary.md
- reports/bugs/bug_api1_registration_content_type_500.md
- test-cases/api2_cart_ai_generated.md
- test-cases/api2_cart_audit.md
- test-cases/api2_cart_human_extension.md
- postman/HW06_API2_Cart.postman_collection.json
- docs/api2_cart_execution_summary.md
- test-cases/api3_access_control_ai_generated.md
- test-cases/api3_access_control_audit.md
- test-cases/api3_access_control_human_extension.md
- postman/HW06_API3_AccessControl.postman_collection.json
- docs/api3_access_control_execution_summary.md
- three API 3 security bug reports
- .github/workflows/api-tests.yml
- postman/HW06_CI_Smoke.postman_collection.json
- postman/HW06_CI.postman_environment.json
- docs/ci_cd_report.md
- test-cases/HW06_Test_Cases_and_Summary_23127531.xlsx

Human responsibility statement

The student remained responsible for running commands, inspecting the SUT, executing Newman/Postman, collecting screenshots, reviewing AI-generated

cases, deciding which observed behaviors were genuine specification defects, publishing GitHub Issues, and committing/pushing the final artifacts.